

USB_ATTACH_DETACH_DESIGN

USB Attach / Detach Handling — Design Document

Project: `com.zebra.rfid.usb`
Target Device: Zebra RFD40 RFID Sled (VID `1504`)
Min SDK: 34 (Android 14)

Overview

Android exposes USB host events via the `UsbManager` system service and broadcasts two key actions:

Action	Constant	When fired
<code>android.hardware.usb.action.USB_DEVICE_ATTACHED</code>	<code>UsbManager.ACTION_USB_DEVICE_ATTACHED</code>	USB device plugged in
<code>android.hardware.usb.action.USB_DEVICE_DETACHED</code>	<code>UsbManager.ACTION_USB_DEVICE_DETACHED</code>	USB device unplugged

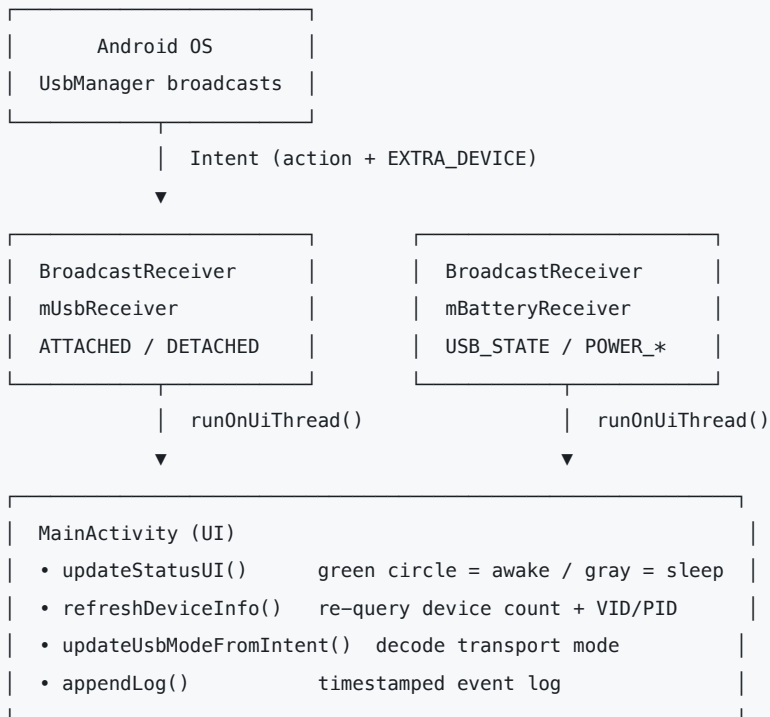
In addition to host attach/detach, the app performs **active USB transport and power-state monitoring** using three more broadcasts:

Action	Constant	When fired
<code>android.hardware.usb.action.USB_STATE</code>	<code>ACTION_USB_STATE</code> (local const)	USB client/transport configuration changes
<code>android.intent.action.ACTION_POWER_CONNECTED</code>	<code>Intent.ACTION_POWER_CONNECTED</code>	Power/USB cable connected
<code>android.intent.action.ACTION_POWER_DISCONNECTED</code>	<code>Intent.ACTION_POWER_DISCONNECTED</code>	Power/USB cable disconnected

The app uses **two** runtime `BroadcastReceiver` s:

- `mUsbReceiver` — host attach/detach events (device count + awake/sleep UI).
- `mBatteryReceiver` — USB transport mode + power connect/disconnect monitoring.

Architecture



Both receivers are registered in `onCreate()` and unregistered in `onDestroy()`.

1. IntentFilter Registration

Register the receiver **programmatically** (not via manifest) so it receives events while the app is in the foreground.

```
IntentFilter filter = new IntentFilter();
filter.addAction(UsbManager.ACTION_USB_DEVICE_ATTACHED); //
    "android.hardware.usb.action.USB_DEVICE_ATTACHED"
filter.addAction(UsbManager.ACTION_USB_DEVICE_DETACHED); //
    "android.hardware.usb.action.USB_DEVICE_DETACHED"

filter.addAction(UsbManager.ACTION_USB_ACCESSORY_ATTACHED);
filter.addAction(UsbManager.ACTION_USB_ACCESSORY_DETACHED);
registerReceiver(mUsbReceiver, filter);
```

Important: Always pair `registerReceiver()` in `onCreate()` with `unregisterReceiver()` in `onDestroy()` to avoid memory leaks.

```
@Override
protected void onDestroy() {
    super.onDestroy();
    unregisterReceiver(mUsbReceiver);
    unregisterReceiver(mBatteryReceiver);
}
```

2. BroadcastReceiver Implementation

```
BroadcastReceiver mUsbReceiver = new BroadcastReceiver() {
    public void onReceive(Context context, Intent intent) {
        String action = intent.getAction();
        if (action == null) return;

        Log.d(TAG, "USB action: " + action);

        // — DETACH —————
        if (UsbManager.ACTION_USB_DEVICE_DETACHED.equals(action)) {
            Log.d(TAG, "USB device detached");

            // EXTRA_DEVICE is available on detach (device already gone from bus)
            UsbDevice device = intent.getParcelableExtra(UsbManager.EXTRA_DEVICE);
            if (device != null) {
                Log.d(TAG, "EXTRA_DEVICE Name=" + device.getDeviceName()
                    + device.getProductName() + device.getDeviceId());
            }

            runOnUiThread(() -> {
                refreshDeviceInfo(); // re-query UsbManager (count will drop to 0)
                updateStatusUI(0);   // → gray circle, sleep hint
            });
        }

        // — ATTACH —————
        if (UsbManager.ACTION_USB_DEVICE_ATTACHED.equals(action)) {
            Log.d(TAG, "ACTION_USB_DEVICE_ATTACHED");

            // On attach, device is in the Intent extras under key "device"
            UsbDevice device = (UsbDevice) intent.getExtras().get("device");
            int vid = (device != null) ? device.getVendorId() : -1;
            Log.d(TAG, "--> Vid =" + vid);

            final int finalVid = vid;
            runOnUiThread(() -> {
                refreshDeviceInfo(); // re-query UsbManager (count now ≥ 1)
                if (finalVid == RFID_VID)
                    updateStatusUI(1); // → green circle, awake state
            });
        }
    }
};
```

3. Initial Device Enumeration on Launch

On startup, query `UsbManager` before registering the receiver to handle the case where the RFD40 is already connected:

```

UsbManager mUsbManager = getSystemService(UsbManager.class);

HashMap<String, UsbDevice> deviceList = mUsbManager.getDeviceList();
int count = deviceList.size();
Log.d(TAG, "Device List Size: " + count);           // e.g. "Device List Size: 1"

for (UsbDevice device : deviceList.values()) {
    Log.d(TAG, "Device VID: " + device.getVendorId()); // e.g. 1504
    Log.d(TAG, "Device PID: " + device.getProductId()); // e.g. 5889
}

updateStatusUI(count); // immediately reflect awake/sleep state

```

4. Status UI Logic

```

private static final int RFID_VID = 1504; // Zebra VID

private void updateStatusUI(int deviceCount) {
    if (deviceCount >= 1) {
        // Green circle – device awake
        mViewStatusCircle.setBackgroundResource(R.drawable.shape_circle_awake);
        mTvStatusIcon.setText("🕒");
        mTvStatusTitle.setText("RFD40 Awake");
        mTvStatusTitle.setTextColor(0xFF2E7D32);
        mTvStatusHint.setText("Device connected and ready");
    } else {
        // Gray circle – sleep / disconnected
        mViewStatusCircle.setBackgroundResource(R.drawable.shape_circle_sleep);
        mTvStatusIcon.setText("zZ");
        mTvStatusTitle.setText("Sleep Mode");
        mTvStatusTitle.setTextColor(0xFF546E7A);
        mTvStatusHint.setText("Hold RFD40 trigger to wake up RFID Reader");
    }
}

```

4.1 Interface-Change Counters

The app tracks how many times the USB interface has changed during the current session via two `int` fields and a derived total, surfaced in the **USB INTERFACE CHANGES** card.

```

private int mAttachCount = 0;
private int mDetachCount = 0;

/** Refreshes the attach/detach/total interface-change counters in the UI. */
private void updateCountersUI() {
    mTvAttachCount.setText(String.valueOf(mAttachCount));
    mTvDetachCount.setText(String.valueOf(mDetachCount));
    mTvTotalCount.setText(String.valueOf(mAttachCount + mDetachCount));
}

```

Each receiver branch increments its counter on the UI thread, logs the running value, and refreshes the card:

```

// DETACH branch
mDetachCount++;
appendLog("Detach count = " + mDetachCount);
updateCountersUI();

// ATTACH branch
mAttachCount++;
appendLog("Attach count = " + mAttachCount);
updateCountersUI();

```

Counter	View id	Increments on	Color
Attach	tv_attach_count	ACTION_USB_DEVICE_ATTACHED	green #2E7D32
Detach	tv_detach_count	ACTION_USB_DEVICE_DETACHED	red #C62828
Total	tv_total_count	mAttachCount + mDetachCount (derived)	blue #0D47A1

Scope: counters are in-memory session values — they start at 0 on launch and are **not** persisted across activity recreation (e.g. rotation). Persist via `onSaveInstanceState` / `SharedPreferences` if cross-session totals are required.

5. Key Log Messages and Their Meaning

```
Device List Size: 1          → RFD40 present on startup
Device VID: 1504            → Zebra vendor confirmed
Device PID: 5889            → RFD40 product ID

USB action: android.hardware.usb.action.USB_DEVICE_DETACHED
USB device detached
EXTRA_DEVICE Name=/dev/bus/usb/004/004  RFD4031-G00B700-E8:::EA4004
Detach count = 1

USB action: android.hardware.usb.action.USB_DEVICE_ATTACHED
ACTION_USB_DEVICE_ATTACHED
Name=/dev/bus/usb/004/005  RFD4031-G00B700-E8:::EA4005  1504
--> Vid =1504
Attach count = 1
```

6. Manifest — USB Device Filter (Optional Deep-Link)

To launch the app automatically when the RFD40 is plugged in, add to `AndroidManifest.xml`:

```
<activity android:name=".MainActivity">
  <intent-filter>
    <action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
  </intent-filter>
  <meta-data
    android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
    android:resource="@xml/usb_device_filter" />
</activity>
```

`res/xml/usb_device_filter.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <usb-device vendor-id="1504" product-id="5889" />
</resources>
```

This is **not required** for the runtime receiver — only needed if you want the OS to auto-launch the app on plug-in.

7. Active USB Transport & Power Monitoring (`mBatteryReceiver`)

Beyond host attach/detach, the app tracks the **USB client transport mode** and **power-cable state** with a second receiver. This distinguishes a *power-only (charging)* cable from a *data-capable (file-transfer / debug)* connection.

7.1 Registration

Registered alongside `mUsbReceiver` in `onCreate()`:

```

IntentFilter usbStateFilter = new IntentFilter();
usbStateFilter.addAction(ACTION_USB_STATE); // "android.hardware.usb.action.USB_STATE"
usbStateFilter.addAction(Intent.ACTION_POWER_CONNECTED);
usbStateFilter.addAction(Intent.ACTION_POWER_DISCONNECTED);
registerReceiver(mBatteryReceiver, usbStateFilter);

// Seed the initial mode from the sticky USB_STATE broadcast, if available.
refreshUsbModeFromStickyState();

```

USB_STATE is a **sticky broadcast**, so the current transport state can be read on demand (e.g. on launch or when power connects) without waiting for a fresh change:

```

private void refreshUsbModeFromStickyState() {
    try {
        Intent stickyUsbState = registerReceiver(null, new IntentFilter(ACTION_USB_STATE));
        if (stickyUsbState != null) {
            updateUsbModeFromIntent(stickyUsbState, "sticky");
        } else {
            Log.d(TAGUSB, "USB_STATE sticky intent unavailable");
        }
    } catch (Exception e) {
        Log.w(TAGUSB, "Failed to refresh USB state from sticky intent", e);
    }
}

```

7.2 Receiver branches

```

BroadcastReceiver mBatteryReceiver = new BroadcastReceiver() {
    public void onReceive(Context context, Intent intent) {
        String action = intent.getAction();
        if (action == null) return;

        if (ACTION_USB_STATE.equals(action)) {
            updateUsbModeFromIntent(intent, "broadcast"); // decode transport mode
            return;
        }

        if (Intent.ACTION_POWER_CONNECTED.equals(action)) {
            refreshUsbModeFromStickyState(); // re-read latest transport state
            String mode = usbFileTransferModeActive
                ? "File Transfer / Debug" : "Power-Only (Charging)";
            // → status text + event log
            return;
        }

        if (Intent.ACTION_POWER_DISCONNECTED.equals(action)) {
            usbConnected = false;
            usbConfigured = false;
            usbFileTransferModeActive = false; // reset decoded state
            // → status text + event log
        }
    }
};

```

7.3 Mode-Determination Logic

`updateUsbModeFromIntent(Intent, String)` decodes boolean extras from the `USB_STATE` intent:

- **Base flags:** `connected`, `configured`
- **Explicit transport functions:** `mtp`, `ptp`, `mass_storage`, `adb`, `rndis`

```

private void updateUsbModeFromIntent(Intent usbStateIntent, String source) {
    usbConnected = usbStateIntent.getBooleanExtra(EXTRA_USB_CONNECTED, false);
    usbConfigured = usbStateIntent.getBooleanExtra(EXTRA_USB_CONFIGURED, false);

    boolean explicitDataModeKnown =
        usbStateIntent.hasExtra(EXTRA_USB_MTP) || usbStateIntent.hasExtra(EXTRA_USB_PTP)
        || usbStateIntent.hasExtra(EXTRA_USB_MASS_STORAGE)
        || usbStateIntent.hasExtra(EXTRA_USB_ADB) || usbStateIntent.hasExtra(EXTRA_USB_RNDIS);

    if (explicitDataModeKnown) {
        usbFileTransferModeActive =
            usbStateIntent.getBooleanExtra(EXTRA_USB_MTP, false)
            || usbStateIntent.getBooleanExtra(EXTRA_USB_PTP, false)
            || usbStateIntent.getBooleanExtra(EXTRA_USB_MASS_STORAGE, false)
            || usbStateIntent.getBooleanExtra(EXTRA_USB_ADB, false)
            || usbStateIntent.getBooleanExtra(EXTRA_USB_RNDIS, false);
    } else {
        // Fallback for vendor builds that omit per-function extras:
        // a connected + configured USB is treated as a data-capable link.
        usbFileTransferModeActive = usbConnected && usbConfigured;
    }
}

```

Condition	Decided Mode
Any of <code>mtp</code> / <code>ptp</code> / <code>mass_storage</code> / <code>adb</code> / <code>rndis</code> is <code>true</code>	File Transfer / Debug (<code>usbFileTransferModeActive = true</code>)
No function extras present, but <code>connected && configured</code>	File Transfer / Debug (fallback inference)
<code>ACTION_POWER_CONNECTED</code> while <code>usbFileTransferModeActive == false</code>	Power-Only (Charging)

Fallback rule: custom OS builds may not broadcast the per-function extras. In that case `connected && configured` is used to infer an active data connection.

8. Lifecycle Summary

```

onCreate()
└─ enumerate existing devices → updateStatusUI(count)
└─ registerReceiver(mUsbReceiver, filter) [ATTACHED/DETACHED]
└─ registerReceiver(mBatteryReceiver, usbStateFilter) [USB_STATE/POWER_*]
└─ refreshUsbModeFromStickyState() [seed initial transport mode]
└─ ensureBluetoothPermissions() [request BLUETOOTH_CONNECT/SCAN on API 31+]
└─ attemptStartupRfidConnection() [if permitted → mRfidHandler.startup()]

onRequestPermissionsResult(BT granted)
└─ mRfidHandler.startup() [deferred startup connect]
└─ (denied) → showRfd40Prompt(...)

onReceive(ATTACHED) [mUsbReceiver]
└─ runOnUiThread → mAttachCount++ → updateCountersUI()
    → refreshDeviceInfo() → updateStatusUI(1) [green]
└─ if VID == RFID_VID → mRfidHandler.onUsbAttached() [cleanup stale → init SDK + connect]

onReceive(DETACHED) [mUsbReceiver]
└─ runOnUiThread → mDetachCount++ → updateCountersUI()
    → refreshDeviceInfo() → updateStatusUI(0) [gray]
└─ mRfidHandler.onUsbDetached() [stop using reader; cleanup deferred]

onReceive(USB_STATE) [mBatteryReceiver]
└─ updateUsbModeFromIntent(intent, "broadcast") → appendLog(summary)

onReceive(POWER_CONNECTED) [mBatteryReceiver]
└─ refreshUsbModeFromStickyState() → status "POWER CONNECTED" + mode

onReceive(POWER_DISCONNECTED) [mBatteryReceiver]
└─ reset usb flags → status "POWER DISCONNECTED"

onDestroy()
└─ unregisterReceiver(mUsbReceiver)
└─ unregisterReceiver(mBatteryReceiver)
└─ mRfidHandler.onDestroy() [dispose SDK + executor]

```

9. USB State Log Messages

Transport/power events are logged under the `MYUSB_STATE` tag:

```

USB Client action: android.hardware.usb.action.USB_STATE
USB_STATE[broadcast] connected=true, configured=true, mtp=false, ptp=false,
    mass_storage=false, adb=true, rndis=false, explicitDataModeKnown=true, fileTransfer=true

USB Client action: android.intent.action.ACTION_POWER_CONNECTED
ACTION_POWER_CONNECTED mode=File Transfer / Debug, connected=true, configured=true, fileTransfer=true

USB Client action: android.intent.action.ACTION_POWER_DISCONNECTED
ACTION_POWER_DISCONNECTED

```

10. RFID SDK Integration (`RFIDHandler`)

The app drives the **Zebra RFID API3 SDK** (`com.zebra.rfid.api3.*`) directly from the USB attach/detach events handled by `mUsbReceiver` . A dedicated `RFIDHandler` owns the reader lifecycle so `MainActivity` only forwards two events plus teardown.

10.1 Trigger points

Trigger	MainActivity wiring	RFIDHandler behavior
App startup (after BT permissions)	<code>mRfidHandler.startup()</code>	<code>onUsbAttached()</code> flow → init + connect; on failure → RFD40 prompt
<code>USB_DEVICE_ATTACHED</code> (VID 1504)	<code>mRfidHandler.onUsbAttached()</code>	clean up stale session → <code>initSdk()</code> → <code>createInstanceAndConnect()</code> → <code>connectReader()</code> → <code>configureReader()</code>
<code>USB_DEVICE_DETACHED</code>	<code>mRfidHandler.onUsbDetached()</code>	stop using the reader (clear flags); SDK cleanup deferred to next attach
Activity <code>onDestroy()</code>	<code>mRfidHandler.onDestroy()</code>	<code>dispose()</code> (disconnect + <code>readers.Dispose()</code> + executor shutdown)

The attach branch is gated on `finalVid == RFID_VID` so unrelated USB peripherals do not trigger an RFID connect attempt. On detach the SDK is intentionally **not** disconnected — the USB endpoint is already gone, so disconnecting there would fault the SDK's serial IO manager on a dead endpoint. The stale session is torn down at the start of the next `onUsbAttached()` instead.

10.2 SDK dependency

The Zebra RFID API3 SDK is a **proprietary** `.aar` that is not bundled in this repo. It must be placed in `app/libs/` , and is pulled in by `app/build.gradle` :

```
dependencies {  
    // Zebra RFID API3 SDK – drop the vendor .aar/.jar files into app/libs/.  
    implementation fileTree(dir: 'libs', include: ['*.aar', '*.jar'])  
    // ...  
}
```

Required Bluetooth permissions and the USB host feature are already declared in `AndroidManifest.xml` .

10.3 Startup + initialize + connect (attach)

On app launch `MainActivity.attemptStartupRfidConnection()` calls `startup()` once the runtime Bluetooth permissions are granted; `startup()` runs the same path as a USB attach. `onUsbAttached()` first disposes any stale `Readers` instance, then (re)initializes and connects:

```

void startup() {
    onUsbAttached();           // same init + connect path as a USB attach
}

void onUsbAttached() {
    resumeRequested = true;
    runOnBackground(() -> {
        if (readers != null) {           // tear down any stale session first
            readers.Dispose();
            readers = null;
            readersAttached = false;
        }
        mainHandler.post(this::initSdk);
    });
}

private void createInstanceAndConnect() {
    readers = new Readers(appContext, ENUM_TRANSPORT.SERVICE_USB);
    availableReaders = readers.GetAvailableRFIDReaderList();
    if (availableReaders == null || availableReaders.isEmpty()) {
        scheduleDiscoveryRetry(0);       // retry via SERVICE_USB (up to 3x)
        return;
    }
    if (resumeRequested) connectReader();
}

private synchronized String connect() {
    reader.connect();
    // a slow first connect implies a stale session → reconnect once to stabilize
    configureReader();                  // addEventsListener + enable handheld/tag events
    return "RFID Connected: " + reader.getHostName();
}

```

When the connect attempt ends without a usable reader (sleed off or unplugged), the handler reports `onRfidConnectionResult(false, ...)` so the UI can prompt the user to turn on or attach the RFD40 (see §10.5).

10.4 Detach — proactive stop, deferred cleanup

On detach the handler clears its run flags and **best-effort stops the SDK's tag-read pipeline** so the serial IO manager is not left actively polling the now-dead USB endpoint. A full `reader.disconnect()` is still **not** performed here (that faults harder on a dead endpoint); the stale session is torn down at the start of the next `onUsbAttached()`.

```

void onUsbDetached() {
    resumeRequested = false;
    connectionInProgress = false;
    // Best-effort stop the read pipeline so the SDK IO manager stops polling the
    // dead endpoint. Every call is guarded because the endpoint is already gone.
    final RFIDReader detached = reader;
    final EventHandler handler = eventHandler;
    runOnBackground(() -> stopReaderQuietly(detached, handler));
    // No reader.disconnect() here – deferred to the next onUsbAttached().
}

private void stopReaderQuietly(RFIDReader r, EventHandler handler) {
    if (r == null) return;
    try { r.Events.setTagReadEvent(false); r.Events.setHandheldEvent(false); }
    catch (Exception ignored) { /* expected on a dead endpoint */ }
    try { if (handler != null) r.Events.removeEventListener(handler); }
    catch (Exception ignored) { /* expected on a dead endpoint */ }
}

```

Unavoidable SDK log on physical yank: the in-flight read that fails the instant the device is pulled is logged *inside* the SDK (RFIDSerialIOMgr E Run ending due to exception / java.io.IOException: Queueing USB request failed) tens of milliseconds **before** the USB_DEVICE_DETACHED broadcast even reaches the app, so that single line cannot be suppressed from app code. The SDK catches it and ends its receive thread; the proactive stop above keeps it to that one benign occurrence.

10.5 Concurrency model

All SDK calls run on a single-threaded background executor; UI/status messages are posted back to the main thread through a `StatusListener`. The listener has two callbacks:

- `onRfidStatus(message)` — appends progress/diagnostic lines to the event log with a `[RFID]` prefix.
- `onRfidConnectionResult(connected, message)` — a terminal connect outcome that drives the status card: a green **RFD40 Connected** state on success, or an amber **RFD40 Not Connected** prompt ("Turn on the RFD40 (hold its trigger to wake it) or attach it via USB") on failure / disconnect.

`volatile` guards prevent duplicate init/connect races:

Guard	Purpose
<code>initializationInProgress</code>	One SDK init at a time
<code>connectionInProgress</code>	One connect attempt at a time
<code>resumeRequested</code>	Connect only while the sled is attached
<code>readersAttached</code>	Tracks the <code>Readers.attach(this)</code> subscription

10.6 SDK reader events

`RFIDHandler` implements `Readers.RFIDReaderEventHandler` and registers an `RfidEventListener`:

- `RFIDReaderAppeared` → connect if `resumeRequested`
- `RFIDReaderDisappeared` → `disconnect()`

- `eventReadNotify` → log tag ID + RSSI
- `eventStatusNotify(DISCONNECTION_EVENT)` → `disconnect()`

10.7 RFID log messages

RFID lifecycle events are logged under the `MYUSB_RFID` tag and mirrored to the in-app event log:

```
[RFID] Initializing RFID SDK...
[RFID] RFID Connected: RFD4031-G00B700-E8 (842 ms)
[RFID] Tag read: E280689400004003... (RSSI -52)
[RFID] No RFID readers found
[RFID] Not connected: No reader available to connect
```

10.8 SDK threading crash guard

The Zebra SDK runs its serial IO on threads the app does not own and cannot wrap in `try/catch`. `RFIDHandler` installs a process-wide `Thread.UncaughtExceptionHandler` that **swallows only known SDK teardown artifacts** and delegates every other throwable to the original handler so genuine crashes still surface:

Artifact	When it occurs
<code>IOException: Queueing USB request failed</code>	receive thread reads a detached (dead) endpoint
<code>InterruptedException / Receive thread interrupted</code>	SDK interrupts the receive thread during teardown
<code>IllegalStateException: Already running</code>	serial IO manager started twice on rapid USB re-enumeration

Matching is done first by message text ("`Queueing USB request failed`", "`Receive thread interrupted`") — independent of the R8-obfuscated stack — and then by the originating SDK class (`SerialInputOutputManager` / `CommonUsbSerialPort`), so the artifact is reliably absorbed across SDK builds. The guard is removed in `onDestroy()`.